NAME: ABDULLAH **ROLL NO: 044**

CLASS: BSAI **SECTION: 3A**

LAB TASK NO: 3

TASK:1

BFS and DFS Manual Trace

A Breadth First Search (BFS)

Queue data structure (FIFO). First in first out.

Step Visited Queue

1 A B, D, C

2 B D, C, E

3 D C, E, F

4 C E, F

5 E F

6 F G

7 G (End)

Order visited = A, B, D, C, E, F, G

Final Path = A > D > F > G

B (DFS)

Queue data structure (LIFO). Last in first out.

Stack shown from bottom up.

Step Visited Stack

1 A C, D, B

2 B C, D, E

3 E C, D, C

4 C C, D, F

5 F C, D, G

6 G C, D (End)

Order visited = A, B, E, C, F, G

Final Path = A > B > E > C > F > G

The trace by hand goes through the nodes in the order of successor. BFS finds the path with the least amount of edges. DFS goes down the first possible branch. If the code provided goes through in the same order of successors and ends when G is processed then it will give the same results.

TASK:2

DLS and IDDFS

```
from collections import deque
graph = {
"A": ["B", "C"],
"B": ["D", "E"],
"C": ["F"],
"D": [],
"E": ["G"],
"F": [],
"G": []
}
def bfs(start, goal):
queue = deque()
queue.append([start])
visited = []
while queue:
path = queue.popleft()
node = path[-1]


if node == goal:
```

```
return path
if node not in visited:
visited.append(node)
for next_node in graph[node]:
new_path = path + [next_node]
queue.append(new_path)
def dfs(start, goal):
stack = [[start]]
visited = []
while stack:
path = stack.pop()
node = path[-1]
if node == goal:
return path
if node not in visited:
visited.append(node)
for next_node in reversed(graph[node]):
new_path = path + [next_node]
stack.append(new_path)
print("BFS Path:", bfs("A", "G"))
print("DFS Path:", dfs("A", "G"))
```

B) DLS RESULT:

DLS Results

Depth Node Counted: 0.

Depth Status Path

1 CUTOFF N/A

2 CUTOFF N/A

3 FOUND A > D > F > G

4 FOUND A > D > F > G

C. IDDFS Results

Max Depth Limits Tried Status

1 0,1 CUTOFF

2 0,1,2 CUTOFF

3 0,1,2,3 FOUND

4 0,1,2,3 FOUND

D. Explanation

1. Depth 1 and Depth 2 have failed to find the goal as it is located at depth 3.

2. Depth 3 has found the goal.

3. Depth 4 has found the goal but will stop searching at depth 3.

4. IDDFS searches previously searched depths as it goes back to node A with an increased limit.

There are three statuses:

FOUND – The goal has been reached.

CUTOFF – The depth limit has been reached.

FAILURE – There is no solution available from the remaining search paths.

TASK:3

The UCS finds the path with the least overall cost.

A. Weighted Graph

Edges and Weights:

A to B = 11

A to D = 7

A to C = 14

B to E = 15

E to C = 8

C to F = 10

D to F = 25

F to G = 20

E to H = 9

H to G = 10

Extra Corridor: E to H to G

B) UCS Python Code

```
import heapq
graph = {
'A': [('B' 11) ('D' 7) ('C' 14)]
'B': [('E' 15)]
'D': [('F' 25)]
'C': [('F' 10)]
'E': [('C' 8) ('H' 9)]
'F': [('G' 20)]
'H': [('G' 10)]
'G': []
}
def ucs(start, goal):
queue = [(0, start, [start])]
visited = set()
while queue:
cost, node, path = heapq.heappop(queue)
if node in visited:
continue
visited.add(node)
print("Processed:" node, "Cost:" cost)
if node == goal:
return path cost
for child, weight in graph[node]:
if child not in visited:
heapq.heappush(
queue
(cost + weight, child, path + [child]))
```

return [] 0

path cost = ucs('A' 'G')

print("Final Path:" path)

print("Total Cost:" cost)

C UCS Processing Order

Step Processed Node Total Cost

1 A 0

2 D 7

3 B 11

4 C 14

5 F 24

6 E 26

7 H 35

8 G 44

UCS Results

Path: A -> C -> F -> G

Cost:

14 + 10 + 20 = 44

D) BFS and UCS Results

Algorithm Path Cost

BFS A - D -F - G 52

UCS A -C -F -> G 44

BFS finds the first shortest path by looking at the order of the successors. UCS finds the least cost path.

E. Why There Could Be A Longer Path With Less Cost.

BFS looks at how many edges there are.

UCS looks at how much weight the edges have.

The path from BFS had a cost of 52.

The path from UCS had a cost of 44.

So even though there are less edges it can be a more expensive path.

GITHUB LINK:

https://github.com/abdullahrehman957-lgtm/AI-LAB-TASK
